Aim:
Create employee salary program using NodeJS

Objective:
Develop a command-line application that allows for the management of employee records, including the calculation and tracking of salaries, using Node.js and JSON files for data storage.

Technologies Used

- Node.js: JavaScript runtime environment for server-side development.
- File System (fs) Module: Built-in Node.js module for interacting with the file system.
- Readline Module: Built-in Node.js module for reading input from the command line.

Step-by-Step Implementation

1. Initialize the Project

```
mkdir employee-salary-system
cd employee-salary-system
npm init -y
```

2. Create the Data Directory and File

```
mkdir data
echo "[]" > data/employees.json
```

3. Develop the Application (app.js)

```
const fs = require('fs');
```

```
const readline = require('readline');
const path = require('path');

const dataFilePath = path.join(__dirname, 'data', 'employees.json');

// Function to read employee data from the JSON file
function readEmployeeData() {
  try {
    const data = fs.readFileSync(dataFilePath, 'utf8');
    return JSON.parse(data);
  } catch (err) {
    console.error('Error reading employee data:', err);
    return [];
  }
}

// Function to write employee data to the JSON file
function writeEmployeeData(data) {
  try {
    fs.writeFileSync(dataFilePath, JSON.stringify(data, null, 2));
  } catch (err) {
    console.error('Error writing employee data:', err);
  }
}

// Function to calculate salary
function calculateSalary(hourlyRate, hoursWorked) {
  return hourlyRate * hoursWorked;
}

// Function to display the main menu
function displayMenu() {
  console.log('\nEmployee Salary Management System');
  console.log('1. Add Employee');
  console.log('2. View Employees');
  console.log('3. Update Employee');
  console.log('4. Delete Employee');
  console.log('5. Exit');
  rl.question('Enter your choice: ', handleMenu);
}

// Function to handle menu choices
function handleMenu(choice) {
  switch (choice) {
    case '1':
      addEmployee();
      break;
    case '2':
      viewEmployees();
```

```
      break;
    case '3':
      updateEmployee();
      break;
    case '4':
      deleteEmployee();
      break;
    case '5':
      rl.close();
      break;
    default:
      console.log('Invalid choice. Please try again.');
      displayMenu();
  }
}

// Function to add a new employee
function addEmployee() {
  rl.question('Enter employee name: ', (name) => {
    rl.question('Enter position: ', (position) => {
      rl.question('Enter hourly rate: ', (hourlyRate) => {
        rl.question('Enter hours worked: ', (hoursWorked) => {
          const employees = readEmployeeData();
          const id = employees.length ? employees[employees.length - 1].id + 1 : 1;
          const salary = calculateSalary(parseFloat(hourlyRate), parseFloat(hoursWorked));
          employees.push({ id, name, position, hourlyRate: parseFloat(hourlyRate), hoursWorked:
parseFloat(hoursWorked), salary });
          writeEmployeeData(employees);
          console.log('Employee added successfully.');
          displayMenu();
        });
      });
    });
  });
}

// Function to view all employees
function viewEmployees() {
  const employees = readEmployeeData();
  if (employees.length === 0) {
    console.log('No employee records found.');
  } else {
    console.log('\nEmployee Records:');
    employees.forEach((emp) => {
      console.log(`ID: ${emp.id}, Name: ${emp.name}, Position: ${emp.position}, Hourly Rate:
${emp.hourlyRate}, Hours Worked: ${emp.hoursWorked}, Salary: ${emp.salary}`);
    });
  }
  displayMenu();
```

```
}

// Function to update an existing employee
function updateEmployee() {
  rl.question('Enter employee ID to update: ', (id) => {
    const employees = readEmployeeData();
    const employee = employees.find((emp) => emp.id === parseInt(id));
    if (!employee) {
      console.log('Employee not found.');
      displayMenu();
    } else {
      rl.question(`Enter new name (${employee.name}): `, (name) => {
        rl.question(`Enter new position (${employee.position}): `, (position) => {
          rl.question(`Enter new hourly rate (${employee.hourlyRate}): `, (hourlyRate) => {
            rl.question(`Enter new hours worked (${employee.hoursWorked}): `, (hoursWorked) => {
              employee.name = name || employee.name;
              employee.position = position || employee.position;
              employee.hourlyRate = hourlyRate ? parseFloat(hourlyRate) : employee.hourlyRate;
              employee.hoursWorked = hoursWorked ? parseFloat(hoursWorked) :
employee.hoursWorked;
              employee.salary = calculateSalary(employee.hourlyRate, employee.hoursWorked);
              writeEmployeeData(employees);
              console.log('Employee updated successfully.');
              displayMenu();
            });
          });
        });
      });
    }
  });
}

// Function to delete an employee
function deleteEmployee() {
  rl.question('Enter employee ID to delete: ', (id) => {
    let employees = readEmployeeData();
    const index = employees.findIndex((emp) => emp.id === parseInt(id));
    if (index === -1) {
      console.log('Employee not found.');
    } else {
      employees.splice(index, 1);
      writeEmployeeData(employees);
      console.log('Employee deleted successfully.');
    }
    displayMenu();
  });
}

// Create readline interface
```

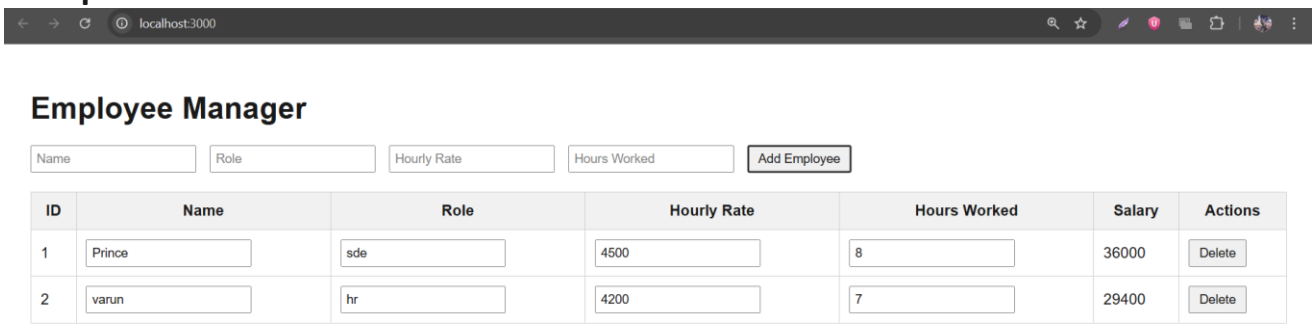
```
const rl = readline.createInterface({
  input: process.stdin,
  output: process.stdout,
});

// Start the application
displayMenu();
```

Testing the Application

1. Run the Application:
   node app.js
2. Interact with the Menu:
   Follow the on-screen prompts to add, view, update, or delete employee records.

Output:




```
PS E:\backend lab\employee-salary> npm install express

up to date, audited 68 packages in 2s

14 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
PS E:\backend lab\employee-salary> node app.js
✅ Server running at http://localhost:3000
```

Learning Outcomes:

a. Learned how to set up routes (GET, POST, PUT, DELETE) using Express to manage CRUD operations for employee data.

b. Gained hands-on experience in reading from and writing to JSON files for persistent data storage without a database.

c. Understood how JSON is used to structure data and how to handle incoming client requests with middleware (express.json()).